



Indian Institute of Technology Guwahati

Department of Electronics and Electrical Engineering

Digital IC Design Laboratory

Lab Report

Course Code: EE518

Semester: 1

Submitted By: Ameya S Moghe

Roll Number: 254102404

EXPERIMENT-05

Aim:

1) Design and implementation of the Radix 4 Booth Multiplier

2) Optimization of the Radix 4 Booth multiplier

Calculate power, number of transistors and hardware utilization

Theory:

The Radix-4 Booth Multiplier is an efficient algorithm used for binary multiplication, designed to reduce the number of partial products and thereby speed up the multiplication process. It is an extension of the original Booth's algorithm (Radix-2) that groups the multiplier bits in overlapping sets of three, allowing the algorithm to handle two bits of the multiplier at a time.

Working Principle

The multiplier bits are examined in overlapping triplets, where each triplet consists of two current bits and one preceding bit (initially zero for the least significant bits). Each triplet determines one partial product to be generated based on the following logic:

- The partial product can be 0, ± 1 , or ± 2 times the multiplicand.
- Negative values are handled using two's complement to simplify hardware design.
- By processing two bits at a time rather than one, the number of partial products is approximately halved compared to a Radix-2 multiplier.

Key Steps in the Algorithm

1. Preprocessing: The multiplier's sign bit is extended if necessary, and a zero bit is appended to its least significant end.
2. Grouping: The multiplier bits are scanned in overlapping groups of three bits.

3. Partial Product Generation: Each group of three bits is used to select a partial product as multiples of the multiplicand according to a predefined encoding table.
4. Shifting: Each partial product is shifted left by a multiple of two bits, corresponding to the position of the processed group in the multiplier.
5. Summation: All partial products are summed to obtain the final product.

Advantages

- Significant reduction in the number of partial products leads to faster multiplication.
- Simplified hardware implementation due to fewer additions.
- Efficient handling of signed numbers as negative partial products are managed uniformly.
- Commonly used in high-speed digital arithmetic units such as in processors and digital signal processors.

Example of Partial Product Encoding

Triplet Bits Operation on Multiplicand

000 or 111 0

001 or 010 $+1 \times$ multiplicand

011 $+2 \times$ multiplicand

100 $-2 \times$ multiplicand

101 or 110 $-1 \times$ multiplicand

This table determines how each triplet in the multiplier translates into multiplying the multiplicand by these factors.

Tools Used: Cadence Virtuoso is a comprehensive electronic design automation (EDA) tool widely used for designing and simulating integrated circuits (ICs), especially analog, mixed-signal, and custom digital circuits. It provides an integrated environment for schematic capture, circuit simulation, and layout design, enabling engineers to create full-custom IC designs efficiently.

1) Design and implementation of the Radix 4 Booth Multiplier

Booth Encoder Block

The encoder which is used in Booth multiplication algorithm is called Booth encoder. It is the

encoder which is used to perform Booth encoding. It performs encoding of 3 input signals.

They are: P2, P1, P0. We will have total 4 Booth encoders. Each of these encoders take inputs

from the multiplier bus, X [0:7]. The number of input bits is 3. The encoder actually takes the 3

inputs as a group. These groupings we have mentioned earlier. The output of the encoder also

consists of 3 bits. The outputs are: S, D, N.

S: If Y is multiplied with + or – 1, S will be 1. Otherwise 0.

D: If Y is multiplied with + or – 2, S will be 1. Otherwise 0.

N: If Y is multiplied with -1 or -2 then N will be 1. If multiplied with 1 or 2 then N will be 0

$$S = P1 \text{ Xor } P0$$

$$D = P2 \cdot \overline{P1} + P2 \cdot \overline{P0}$$

$$N = \overline{P2} \cdot P1 \cdot P0 + P2 \cdot \overline{P1} \cdot \overline{P0}$$

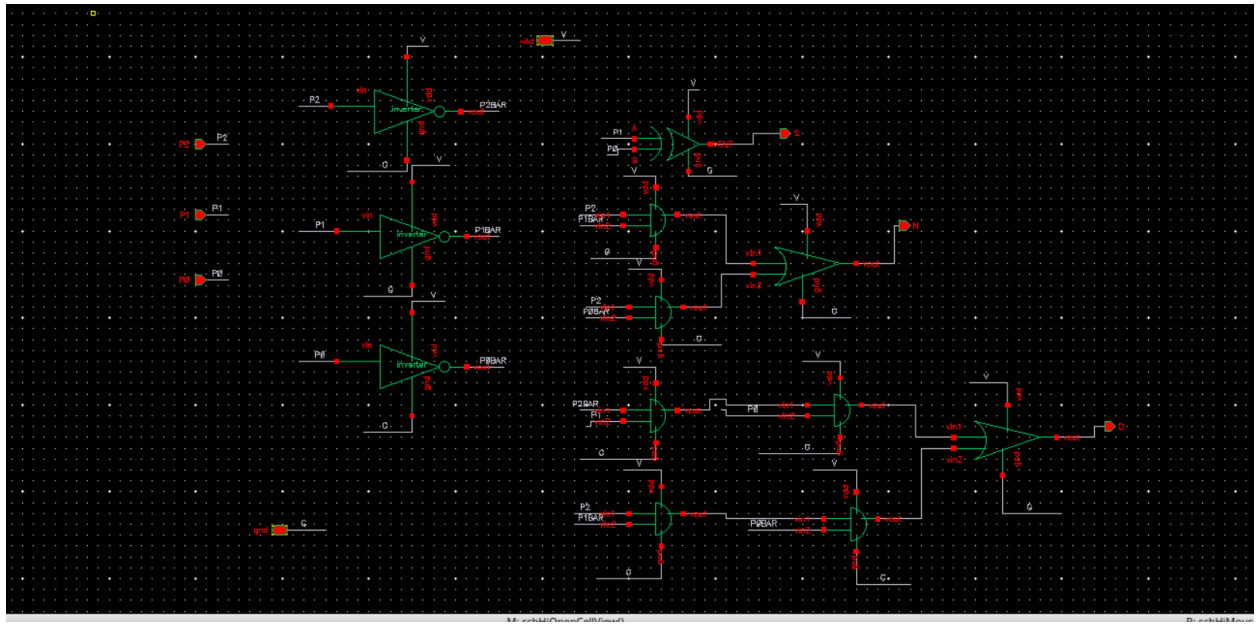


Figure 1.1 Booth Encoder

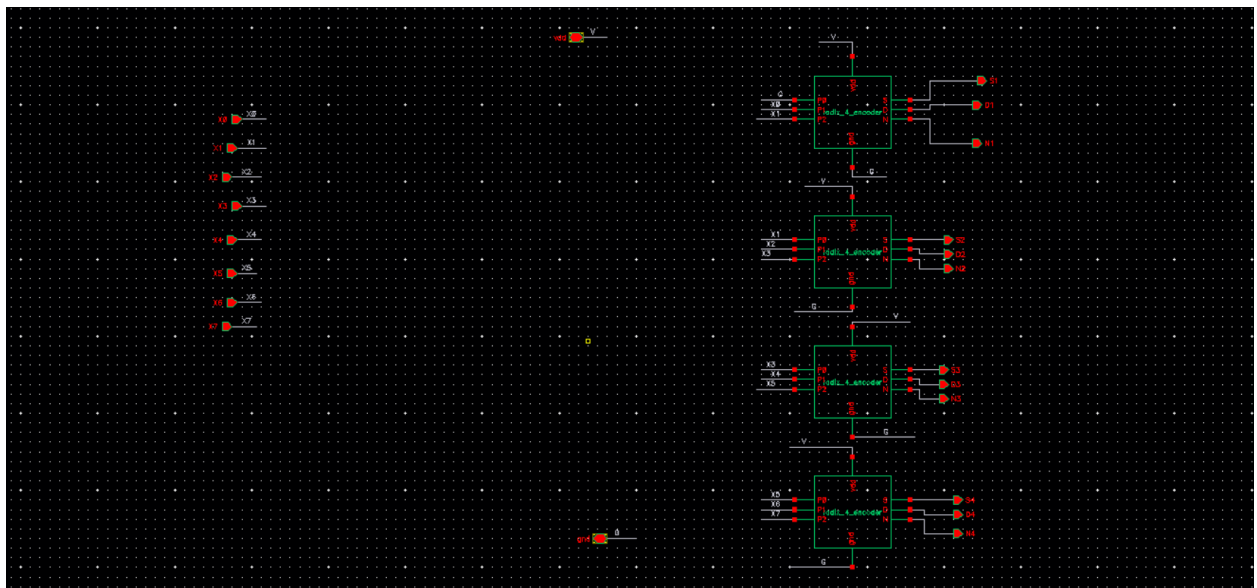


Figure 1.2 8bit Booth Encoder

Booth Decoder Block

This decoder is driven by the outputs of the Booth Encoder blocks. The multiplicand

Y is distributed to all the rows. The select lines of the booth encoders, S, D and N control the

operation of Booth Decoders. This decoder block works in bit by bit operation.

Actually we need to multiply Y by 1 or 2. If Y is multiplied by 1 then Y remains as it is. If we want to multiply by 2 then one is shifted 1 bit and the LSB becomes 0.

When S=1, it gives the present value of Y as output. Multiplied by 1.

When D=1, it gives the previous value of Y as output. Multiplied by 2.

Let's assume a0, a1 are the data bits from Y.

a0 = represents bit of Y

a1 = represents bit of 2Y

Output will depend on value of S, D.

Output= $S \cdot a0 + D \cdot a1$

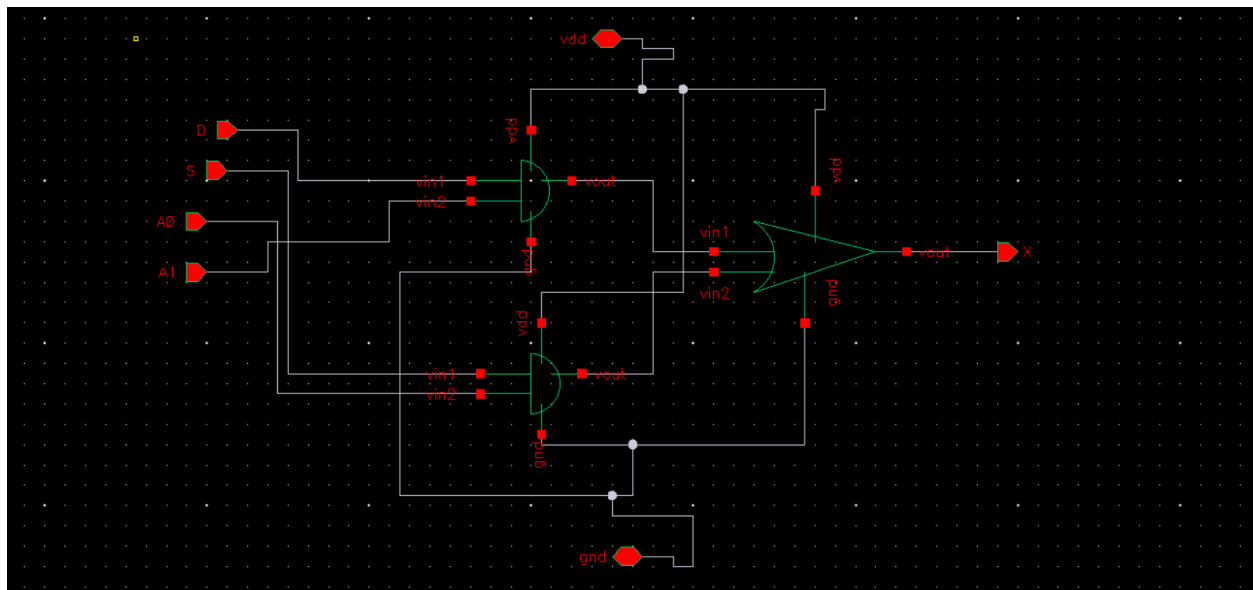


Figure 1.3 Booth Decoder

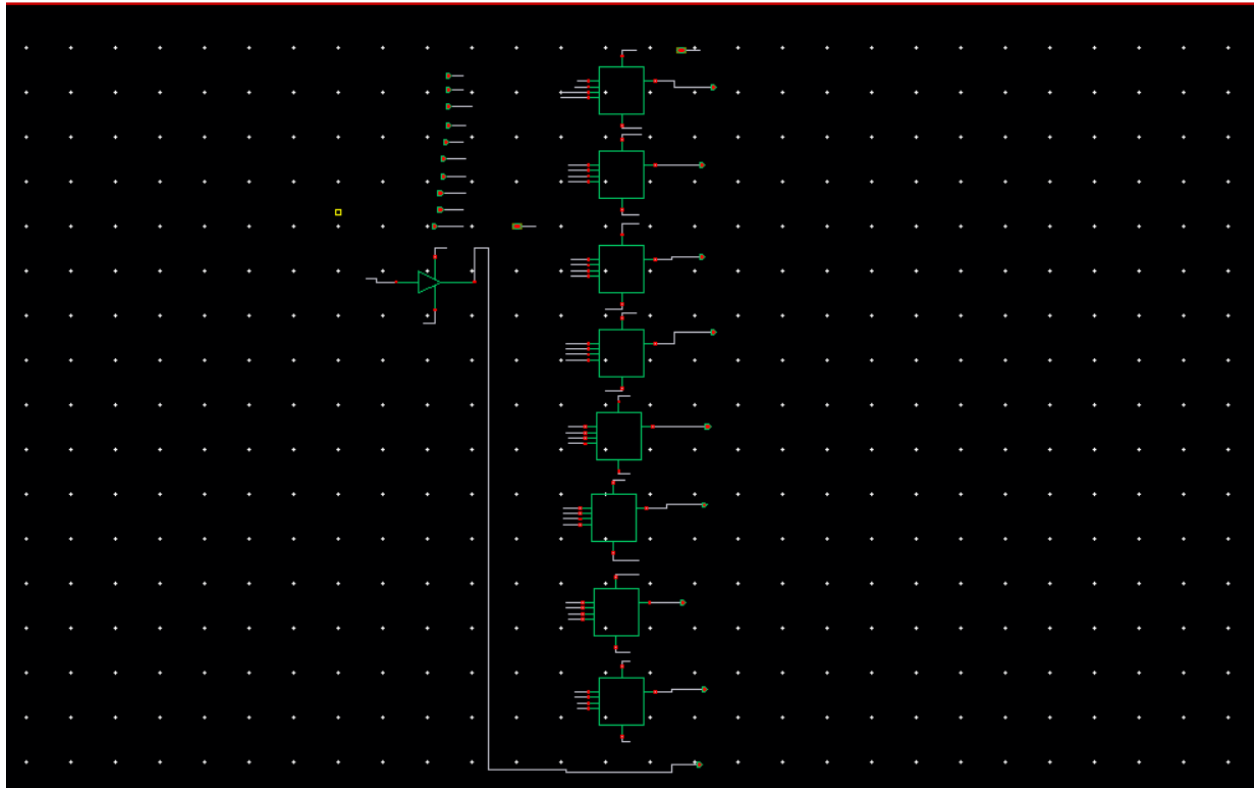


Figure 1.4 8bit Booth Decoder

1's complement:

To do 1's complement those 9 bits from previous blocks were xored with 1.

$D \oplus N$. D is Data and N is output of encoder detects whether the bit is signed or unsigned. When $N=1$ it is treated as negative number when 0 it is positive.

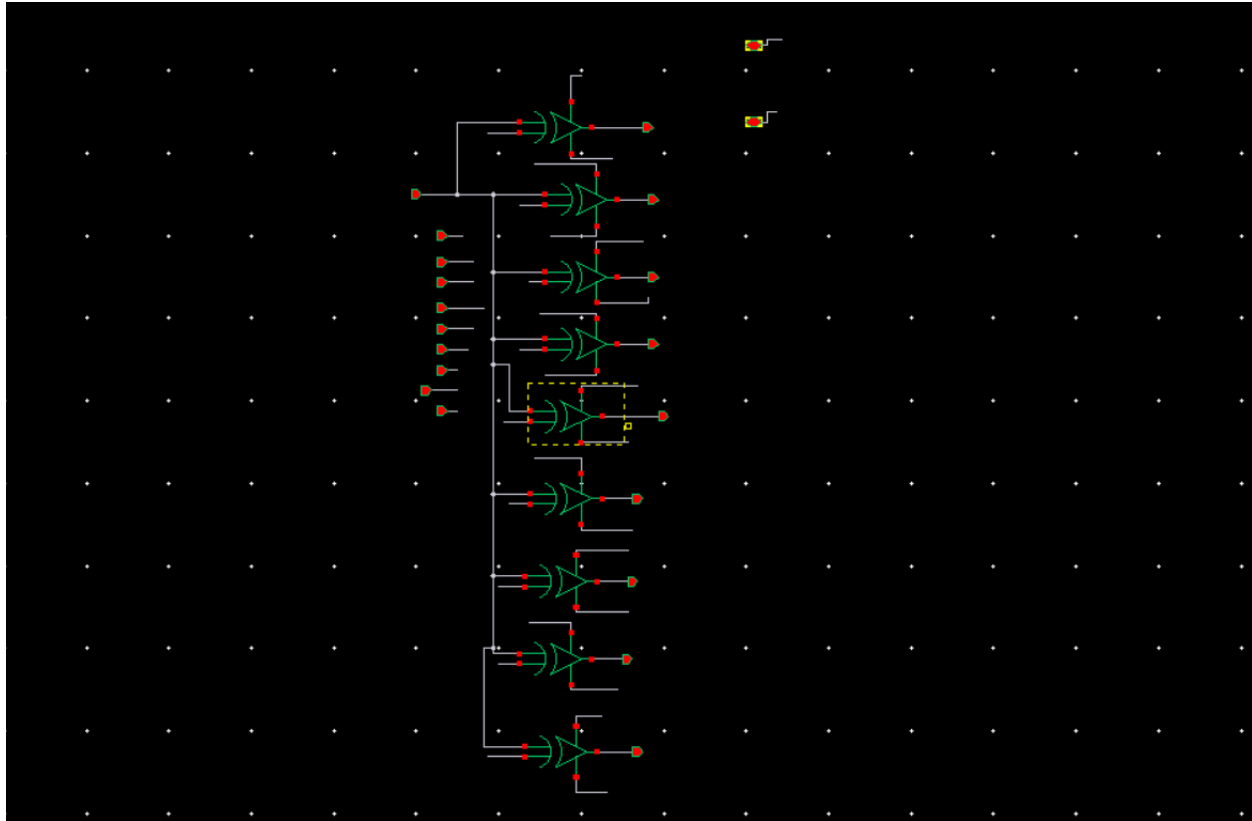


Figure 1.5 1s_Complement

2's complement:

Then that result is added in a 9 bit adder where one input is the output of previous circuit and

another input is 00000000Nb. the result is the 2's complement of Y or $2Y$. The circuit diagram

and description of 9 bit adder will be given in kogge stone adder description section.

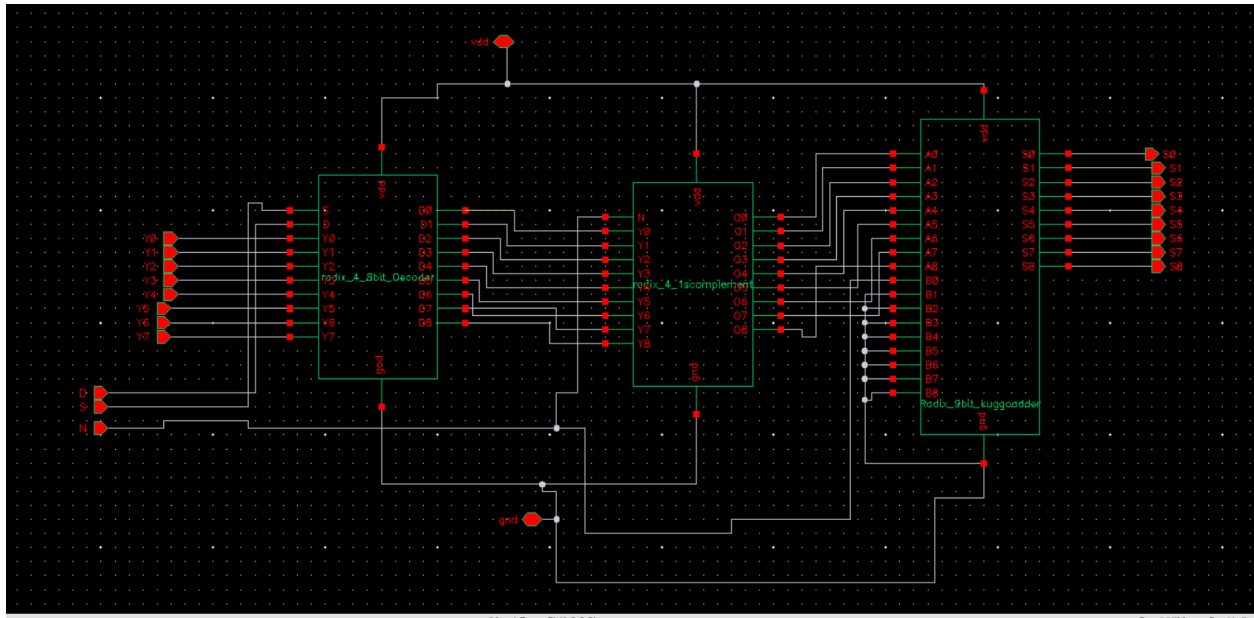


Figure 1.6 Booth Decoder with 2s complement

Combined Decoder and Encoder

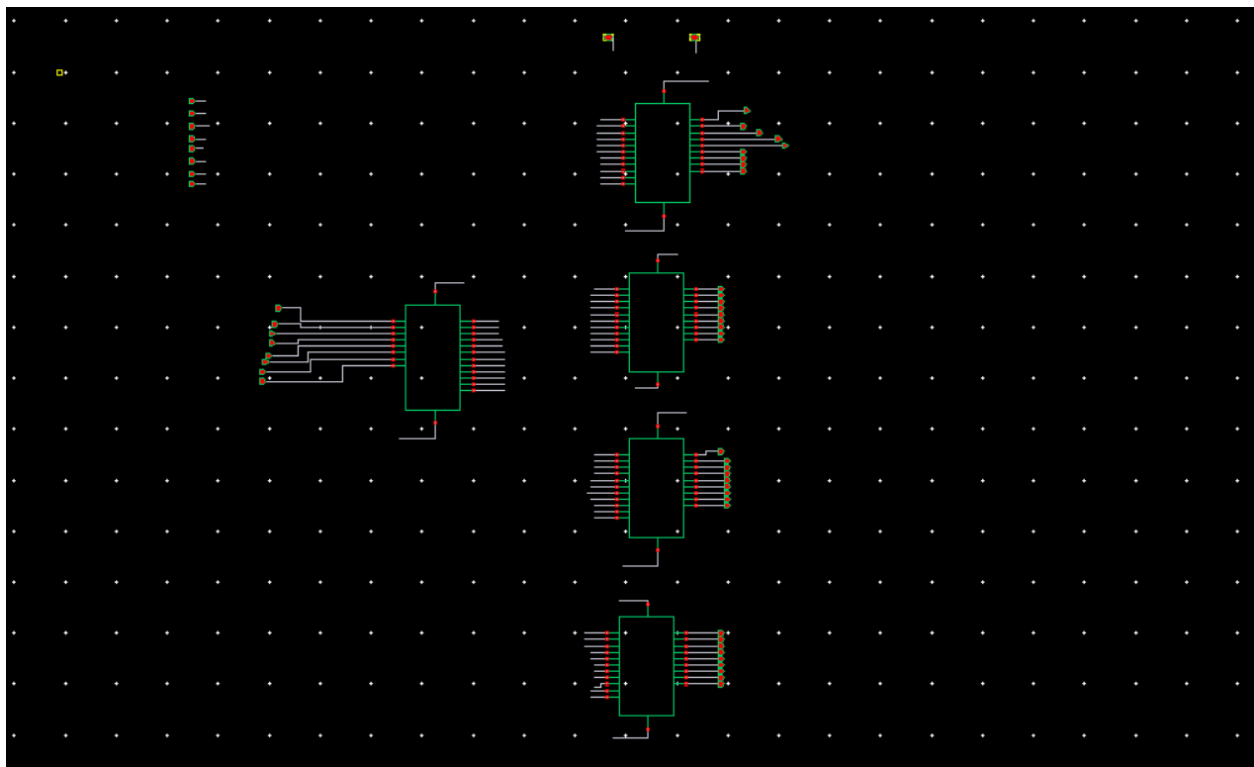


Figure 1.7 Combined Decoder Encoder

Ripple Carry Adder

A 16-bit Ripple Carry Adder (RCA) is a basic digital circuit used to perform binary addition on two 16-bit numbers. It consists of sixteen cascaded 1-bit full adders connected in series, where the carry output of each full adder is fed as the carry input to the next significant bit's adder. This cascading gives rise to the name "ripple carry," as the carry bits ripple through the adder chain from the least significant bit (LSB) to the most significant bit (MSB).

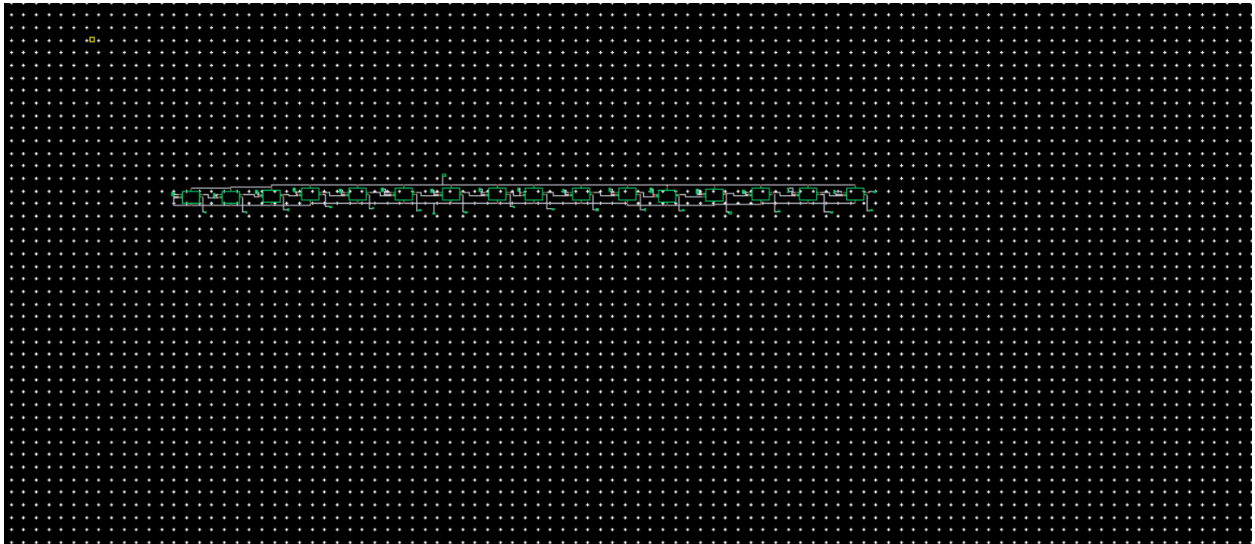


Figure 1.8 Ripple Carry Adder

Multiplier

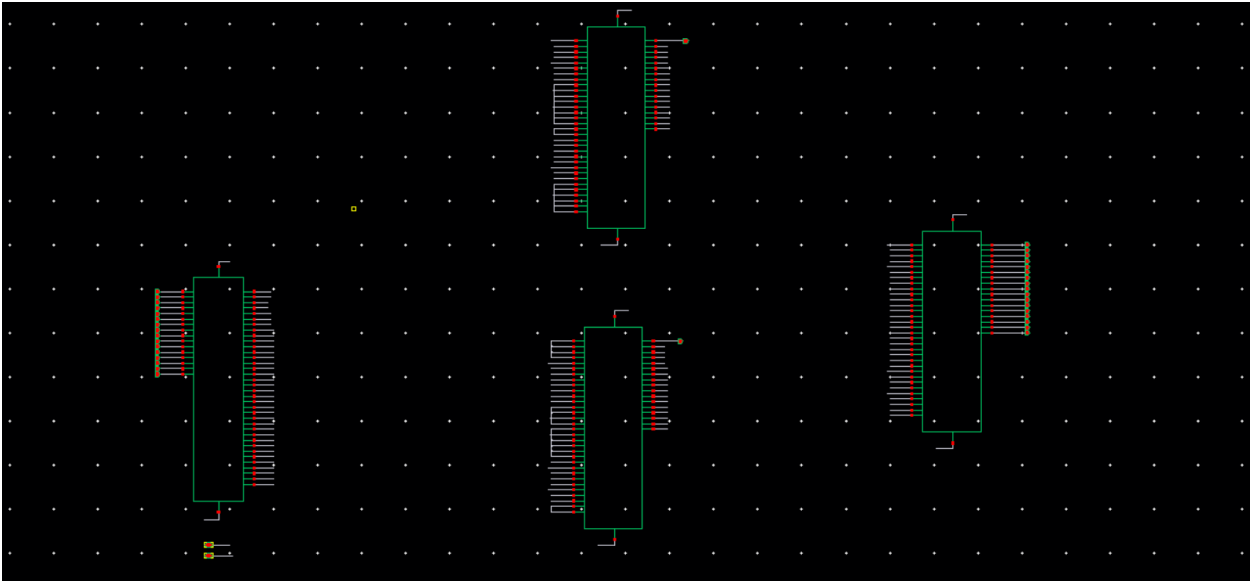


Figure 1.9 Radix_4_Booth_Multiplier

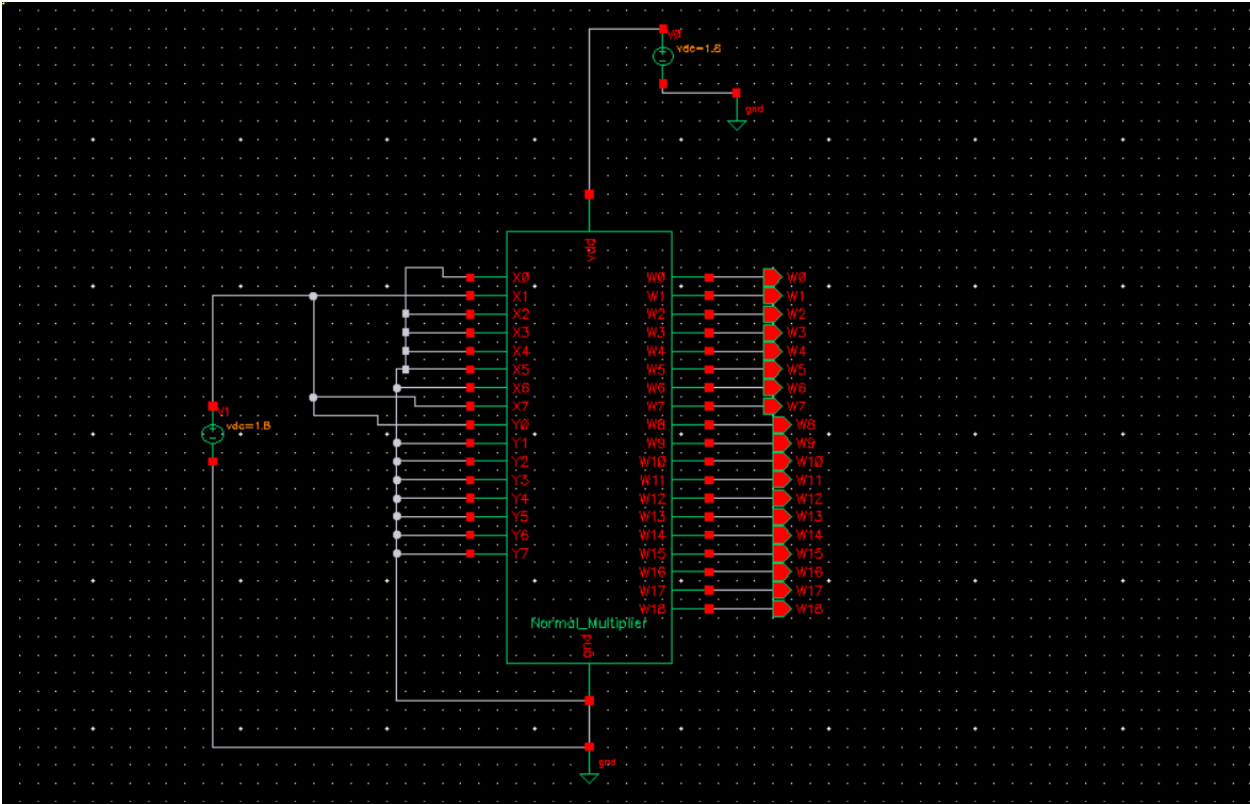


Figure 1.10 Radix_4_Booth_Multiplier

Test Input

X=1000 0010, Y=0000 0001

W=X.Y=1111 1111 0000 0010

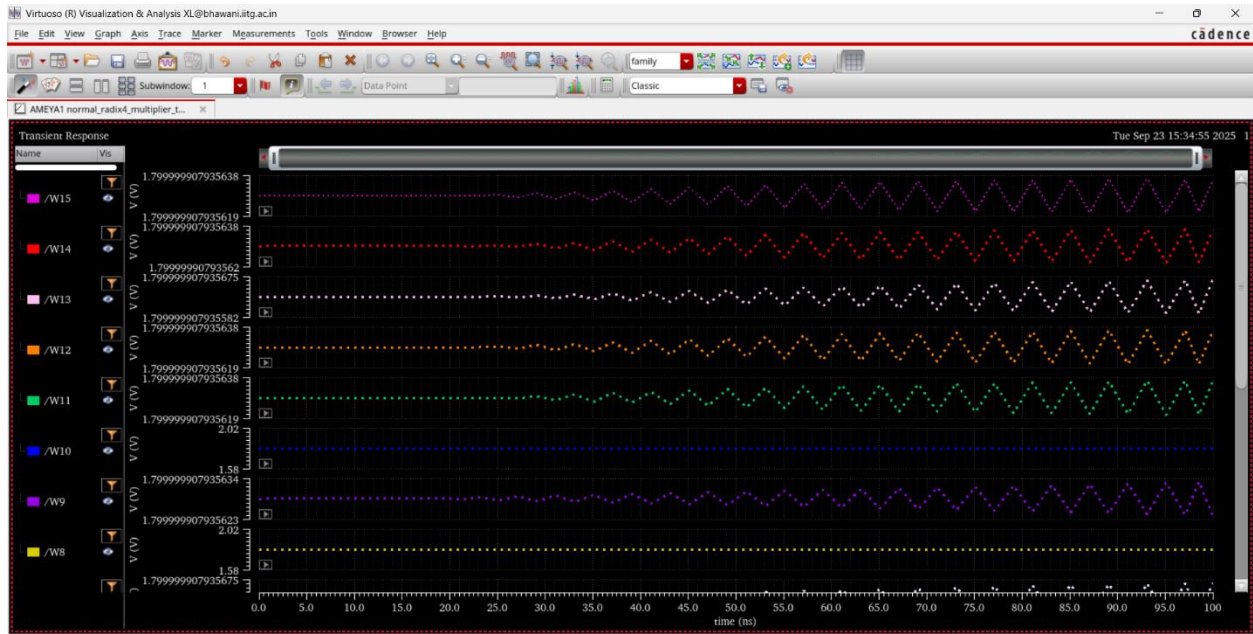


Figure 1.11 Output from MSB to 8th

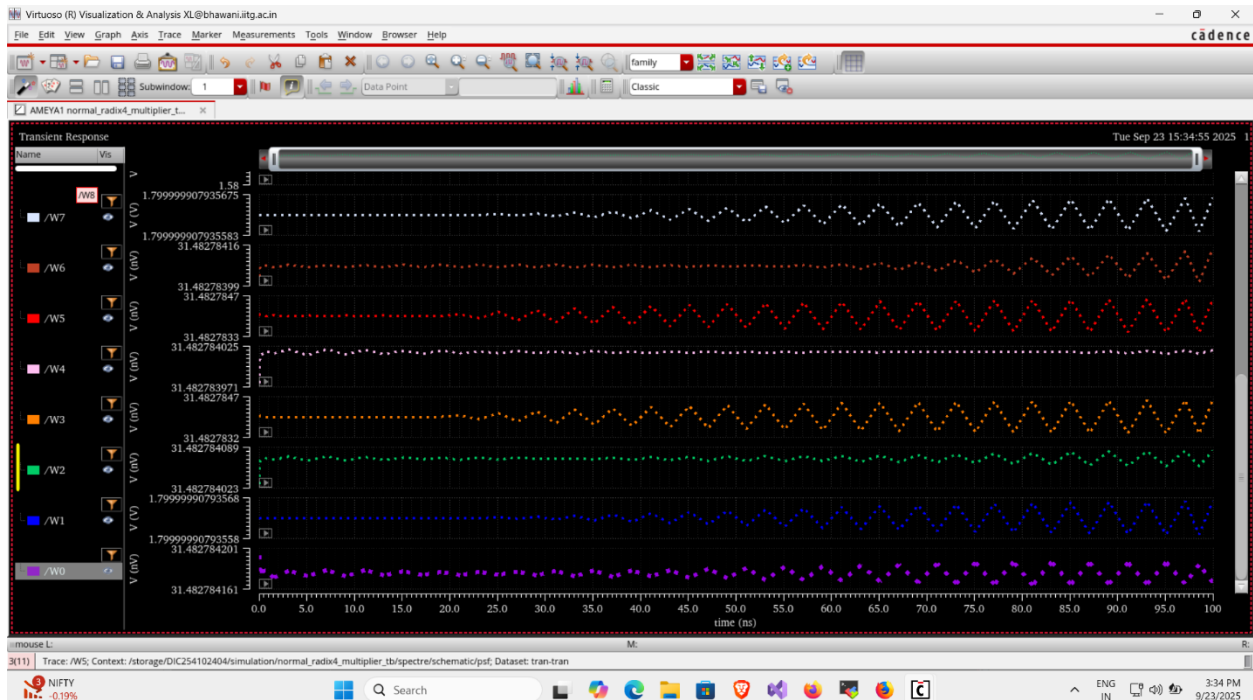


Figure 1.12 Output from 7th to LSB

Power=1.527mw

Number Of Transistors Used=Around 2400 Transistors

Input Voltage Time Period=20ns

$T_{\text{delay}}=2\text{ns}$

$T_{\text{fall}}=1\text{ns}$

$T_{\text{rise}}=1\text{ns}$

$T_{\text{on}}=10\text{ns}$

Delay=1.684 ns

Optimization of the Radix 4 Booth multiplier

In this work, the Radix-4 Booth multiplication algorithm has been optimized by integrating it with a Kogge-Stone adder for partial product addition. The traditional Radix-4 Booth multiplier reduces the number of partial products by encoding the multiplier bits in groups of three, effectively processing two bits per cycle. However, the critical path delay in such multipliers is often dominated by the addition of partial products.

Kogge-Stone Adder

The Kogge-Stone Adder (KSA) is a type of parallel prefix adder widely used for high-speed binary addition in digital arithmetic circuits. It is renowned for offering the shortest possible carry computation delay among common adder architectures, making it ideal for performance-critical applications such as processors and digital signal processing units.

Procedure:

A three step process is generally involved in the construction of a Parallel Prefix Adder. The first

step involves the creation of generate and propagate signals for the input operand bits. The second step involves the generation of carry signals. In the final step, the sum bits of the adder following stages of the operand bits and the preceding stage carry bit using a xor gate.

1. Pre-processing stage.
2. Carry look ahead stage.
3. Post processing stage.

Preprocessing

This step involves computation of generate and propagate signals corresponding to each pair

of bits in A and B.

$$P_i = A_i \text{ xor } B_i$$

$$G_i = A_i \text{ and } B_i$$

Carry Generation Network

In this stage we compute carries corresponding to each bit. Execution of these operations is

carried out in parallel. After the computation of carries in parallel they are segmented into

smaller pieces. It uses carry propagate and generate as intermediate signals which are given

by below equations.

$$P_{P_i:j} = P_{i:k+1} + P_{k:j}$$

$$G_{G_i:j} = G_{i:k} + P_{i:k+1} \cdot G_{k:j}$$

Post Processing

This is the final step and is common to all adders of this family (carry look ahead). It involves

computation of sum bits.

$$C_{i-1} = P_i C_{in} + G_i$$

Sum generation

$$S_i = P_i \text{ xor } C_{i-1}$$

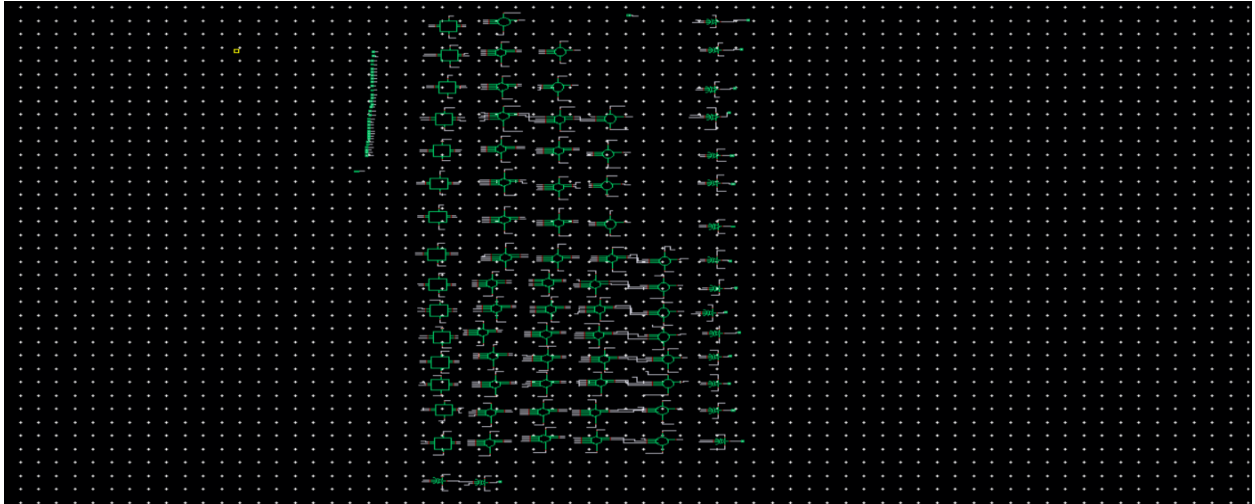


Figure 2.1 Kogge Stone Adder 16 bit

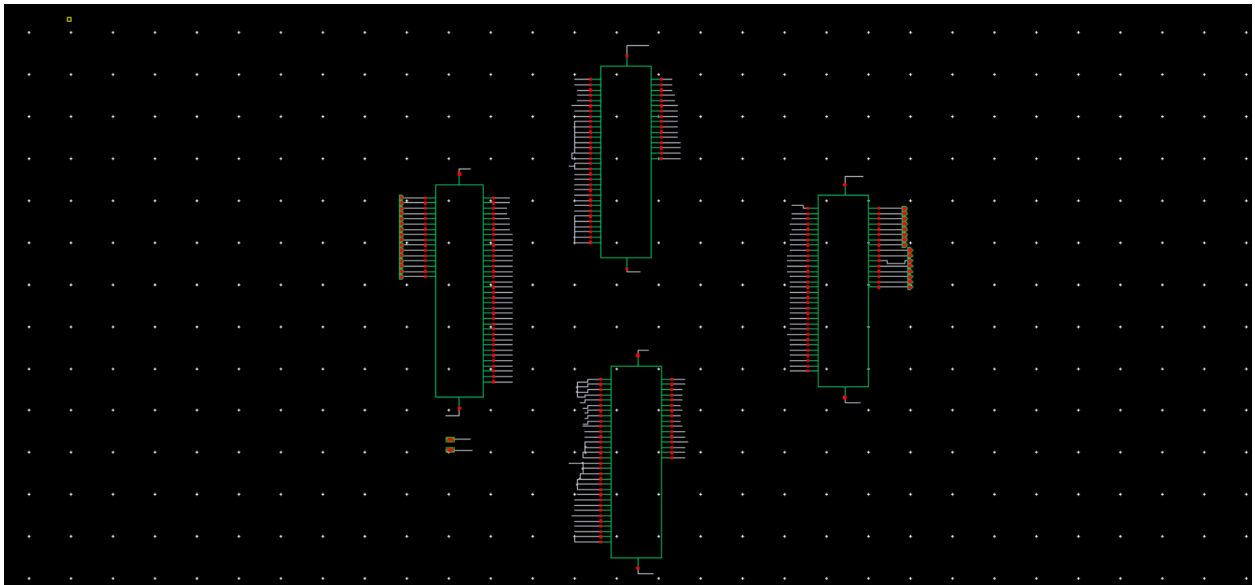


Figure 2.2 Radix_4_Booth Optimized Multiplier

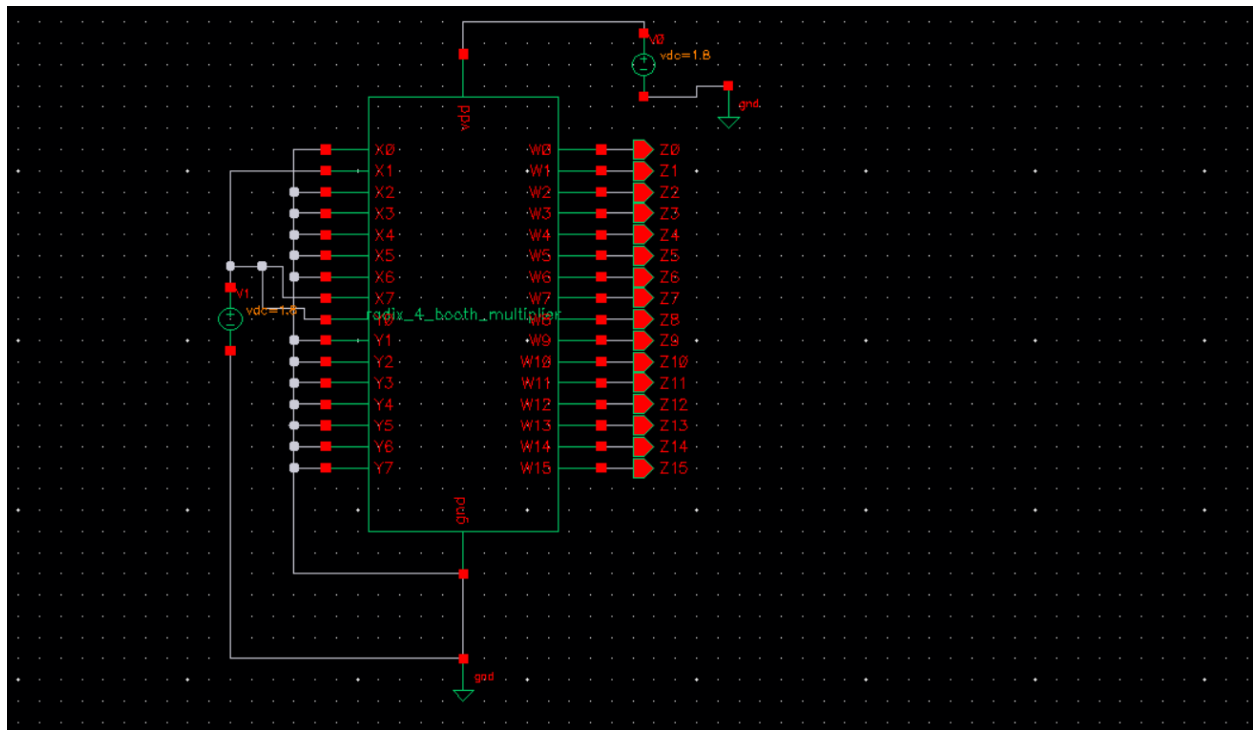


Figure 2.3 Radix_4_Booth Optimized Multiplier

Test Input

$X=1000\ 0010$, $Y=0000\ 0001$

$W=X.Y=1111\ 1111\ 0000\ 0010$

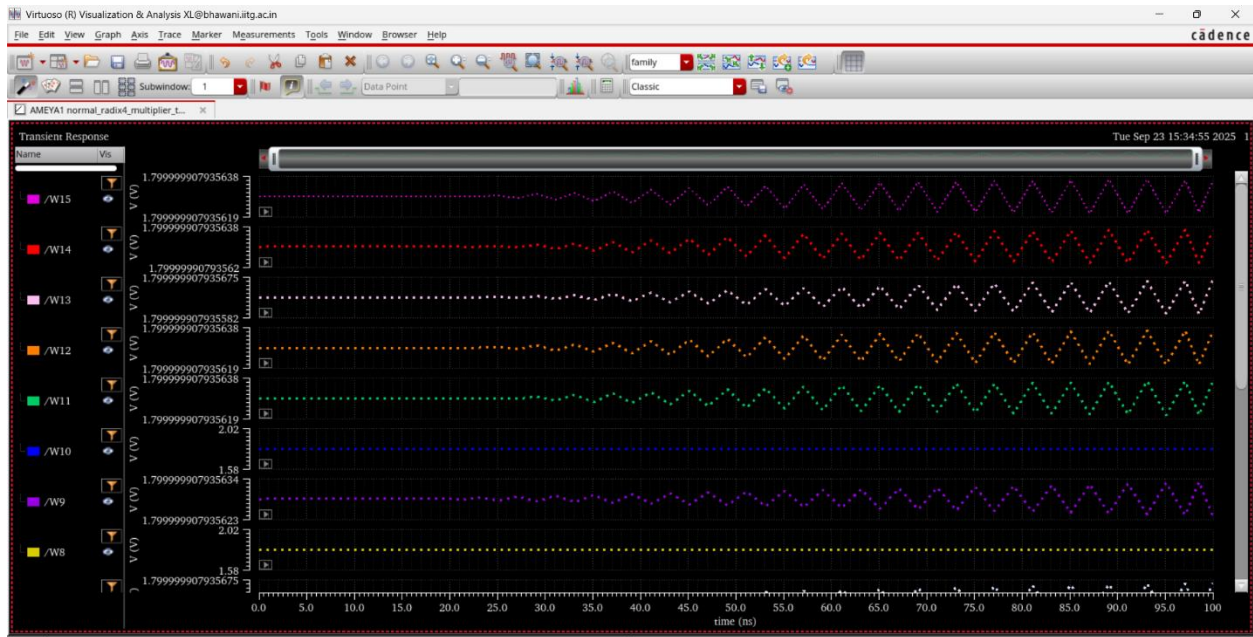


Figure 2.4 Output from MSB to 8th

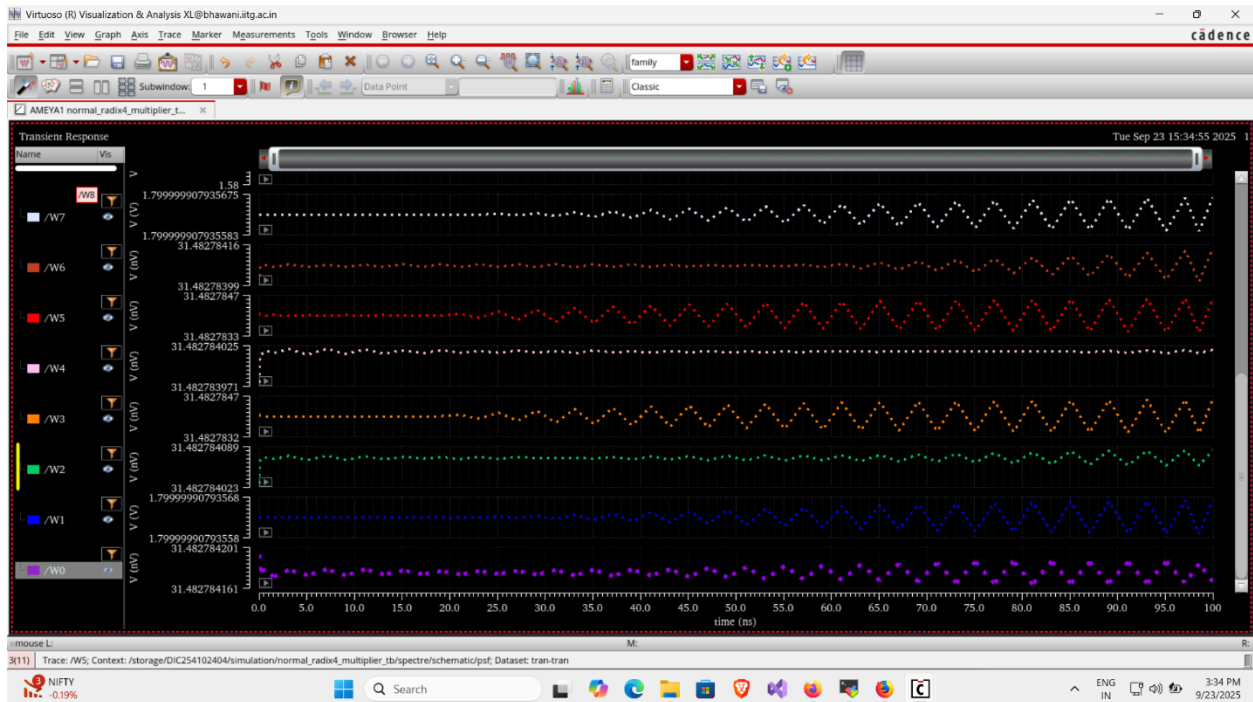


Figure 2.5 Output from 7th to LSB

Power=2.716 mw

Number Of Transistors Used=Around 1800 transistors

Input Voltage Time Period=20ns

$T_{\text{delay}}=2\text{ns}$

$T_{\text{fall}}=1\text{ns}$

$T_{\text{rise}}=1\text{ns}$

$T_{\text{on}}=10\text{ns}$

Delay=1.4214 ns

Result And Discussion

The Radix 4 Booth Multiplier and optimized Radix 4 Booth Multiplier is successfully implemented using cadence virtuoso tool.

The Radix 4 Booth Multiplier consumes less power than optimized one but optimized booth multiplier has less hardware compared to traditional one and has less delay compared to traditional one.

Traditional one is made using Ripple Carry Adder and optimized one is made using Kogge Stone Adder.

Although Power is more in optimized one but we have optimized it in terms of delay and hardware requirements.